



Tokaj-Hegyalja Egyetem
Webkönyvtárak

Készítette: Végh Vencel

Neptunkód: BQ5HU5

Kar: Programtervező informatikus

Tartalom

JAVA DOM DOKUMENTÁCIÓ.....	3
ER Modell dokumentáció	7
XDM Modell dokumentáció	9
UML Diagram dokumentáció	10
XML Dokumentáció	11
XSD Dokumentáció	12

JAVA DOM DOKUMENTÁCIÓ

A Java program célja az XMLBQ5HU5 nevű XML dokumentum beolvasása, feldolgozása, adatainak konzolra történő kiírása, majd a teljes dokumentum mentése egy új XML fájlba. A program a DOM API szabványos elemeit használja, vagyis az XML dokumentumot fa szerkezetként tölti be a memóriába. Ennek köszönhetően a dokumentum elemei, attribútumai és szöveges tartalmai külön objektumként érhetők el.

A program fő osztálya a `DomReadBQ5HU5`, amely a `BQ5HU5` csomagban található. A fájl elején szereplő `package BQ5HU5;` sor azt jelzi, hogy az osztály ehhez a csomaghoz tartozik. Ez azért fontos, mert az IntelliJ projektben a Java fájl a `src/BQ5HU5` mappában helyezkedik el.

A program több Java csomagot importál. Ezek közül a legfontosabbak a DOM API-hoz tartozó importok:

```
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
```

A `Document` az egész XML dokumentumot jelenti. Amikor a program beolvassa az XML fájlt, abból egy `Document` objektum jön létre. Ez a dokumentum gyökere, ezen keresztül érhetők el az XML elemei.

Az `Element` egy konkrét XML elemet jelent. Például az XML-ben szereplő `gondozo`, `allat`, `orokbefogado`, `vizsgalat` vagy `jelentkezes` elemek a Java programban `Element` objektumként kezelhetők.

A `Node` egy általánosabb DOM objektum. A DOM fában minden elem, szöveg, attribútum vagy egyéb szerkezeti rész csomópontként, vagyis `node`-ként jelenik meg. A program először `Node` típusú objektumként olvassa ki az elemeket, majd ha valódi XML elemről van szó, akkor `Element` típusúvá alakítja.

A `NodeList` több XML elem listáját jelenti. Például amikor a program lekéri az összes `allat` elemet, akkor egy `NodeList` objektumot kap vissza, amely tartalmazza az összes állatot.

A programban az XML beolvasásáért a következő rész felel:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();

Document document = builder.parse(inputFile);
```

A `DocumentBuilderFactory` létrehozza azt a gyártó objektumot, amelyből XML feldolgozó készíthető. A `DocumentBuilder` végzi el a tényleges XML beolvasást. A `builder.parse(inputFile)` utasítás beolvassa az XML fájlt, majd létrehozza belőle a DOM

fát. Ettől a ponttól kezdve az XML dokumentum már nem egyszerű szövegfájlként, hanem objektumokból álló szerkezetként kezelhető.

A következő sor a dokumentum szerkezetének rendezésére szolgál:

```
document.getDocumentElement().normalize();
```

A `normalize()` metódus egységesíti a DOM fa szerkezetét. Ez különösen akkor hasznos, ha az XML dokumentumban felesleges sortörések, szóközök vagy szétszakadt szöveges részek vannak. A normalizálás után a program megbízhatóbban tudja feldolgozni az elemeket.

A gyökérelem kiírása így történik:

```
System.out.println("Gyökérelem: " + document.getDocumentElement().getNodeName());
```

A `getDocumentElement()` az XML dokumentum gyökérelemét adja vissza. Ebben a projektben ez az `allatmenhely` elem. A `getNodeName()` metódus ennek az elemnek a nevét adja vissza.

A program külön metódusokban dolgozza fel az XML dokumentum nagyobb egységeit. Ezek a következők:

```
kiirGondozok(document);  
kiirAllatok(document);  
kiirOrokbefogadok(document);  
kiirVizsgalatok(document);  
kiirJelentkezesek(document);
```

Ez a szerkezet átláthatóvá teszi a programot, mert minden metódus egy adott adattípussal foglalkozik. A `kiirGondozok` metódus a gondozók adatait, a `kiirAllatok` az állatok adatait, a `kiirOrokbefogadok` az örökbefogadók adatait, a `kiirVizsgalatok` az orvosi vizsgálatokat, a `kiirJelentkezesek` pedig az örökbefogadási jelentkezéseket írja ki.

Például az állatok feldolgozása így kezdődik:

```
NodeList lista = document.getElementsByTagName("allat");
```

A `getElementsByTagName("allat")` metódus megkeresi az XML dokumentumban az összes `allat` nevű elemet. Az eredmény egy `NodeList`, amely tartalmazza az összes állatot. Ezután a program egy `for` ciklussal végigmegy a listán:

```
for (int i = 0; i < lista.getLength(); i++) {  
    Node node = lista.item(i);
```

A `lista.getLength()` megadja, hány darab elem található a listában. A `lista.item(i)` az aktuális elemet adja vissza. Mivel a DOM API általánosan `Node` objektumként kezeli az elemeket, először ellenőrizni kell, hogy valóban XML elemről van-e szó:

```
if (node.getNodeType() == Node.ELEMENT_NODE) {  
    Element elem = (Element) node;
```

A `Node.ELEMENT_NODE` azt jelenti, hogy az adott csomópont XML elem. Ha ez teljesül, akkor a `Node` objektum `Element` típusú alakítható. Erre azért van szükség, mert az attribútumok és gyermekelemek kényelmes eléréséhez az `Element` típus szükséges.

Az attribútumok olvasása a `getAttribute()` metódussal történik:

```
System.out.println("Állat azonosító: " + elem.getAttribute( name: "id"));  
System.out.println("Gondozó referencia: " + elem.getAttribute( name: "gondozoRef"));
```

Az `id` az adott állat egyedi azonosítója. A `gondozoRef` egy hivatkozás arra a gondozóra, aki az adott állatért felel. Ez a megoldás az XML dokumentumban az egyedek közötti kapcsolatot valósítja meg.

A gyermekelemek szöveges tartalmának kiolvasása a `getText()` segédmetódussal történik:

```
System.out.println("Név: " + getText(elem, tagName: "nev"));  
System.out.println("Faj: " + getText(elem, tagName: "faj"));  
System.out.println("Fajta: " + getText(elem, tagName: "fajta"));  
System.out.println("Kor: " + getText(elem, tagName: "kor"));  
System.out.println("Szín: " + getText(elem, tagName: "szin"));  
System.out.println("Méret: " + getText(elem, tagName: "meret"));
```

A `getText()` metódus azért hasznos, mert nem kell minden elemnél külön leírni ugyanazt a lekérdezési logikát. A metódus megkapja a szülőelemet és annak a gyermekelemnek a nevét, amelynek a tartalmát ki kell olvasni.

A segédmetódus így működik:

```
private static String getText(Element parent, String tagName) { 27 usages  
    NodeList lista = parent.getElementsByTagName(tagName);  
  
    if (lista.getLength() > 0) {  
        return lista.item( index: 0 ).getTextContent();  
    }  
  
    return "";
```

A metódus először megkeresi a szülőelemen belül a megadott nevű gyermekelemet. Ha talál ilyet, akkor visszaadja az első találat szöveges tartalmát. Ha nem talál ilyen elemet, akkor üres szöveget ad vissza. Ez megakadályozza, hogy a program hibával leálljon akkor, ha egy keresett elem nem létezik.

A program a fájlkeresést is külön metódusban végzi. A `keresXmlFajlt()` metódus több lehetséges fájlnevet és elérési utat ellenőriz. Ez azért került bele, mert Windows alatt előfordulhat, hogy a fájl látszólag `XMLBQ5HU5.xml` néven jelenik meg, de valójában más néven vagy más mappában található. A metódus ellenőrzi például az alábbi lehetőségeket:

```
private static File keresXmlFajlt() { 1 usage
    String[] lehetslegesFajlok = {
        "XMLBQ5HU5.xml",
        "XMLBQ5HU5",
        "src/BQ5HU5/XMLBQ5HU5.xml",
        "src/BQ5HU5/XMLBQ5HU5",
        "../XMLBQ5HU5.xml",
        "../XMLBQ5HU5"
    };
};
```

Ha valamelyik fájl létezik, akkor azt adja vissza feldolgozásra. Ha egyik sem található, akkor a program hibaüzenetet ír ki, és nem próbálja meg feldolgozni a hiányzó fájlt.

A program végén a dokumentum mentése történik:

```
File outputFile = new File(inputFile.getParentFile(), child: "BQ5HU5MENTES.xml");
```

Ez a sor létrehozza a kimeneti fájl elérési útját. A mentett fájl neve `XMLBQ5HU51.xml`. A program ugyanabba a mappába menti, ahol a beolvasott XML fájlt megtalálta.

A tényleges mentéshez a program a `Transformer` osztályt használja:

```
TransformerFactory transformerFactory = TransformerFactory.newInstance();
Transformer transformer = transformerFactory.newTransformer();
```

A `TransformerFactory` létrehozza a mentéshez szükséges transzformáló objektumot. A `Transformer` feladata, hogy a memóriában lévő DOM dokumentumot visszaírja XML fájlformátumba.

A következő sorok a mentett XML fájl formázását állítják be:

```
transformer.setOutputProperty(OutputKeys.INDENT, value: "yes");
transformer.setOutputProperty(OutputKeys.ENCODING, value: "UTF-8");
transformer.setOutputProperty(name: "{http://xml.apache.org/xslt}indent-amount", value: "4");
```

Az `INDENT` beállítás gondoskodik arról, hogy az XML fájl tagolt, olvasható formában legyen mentve. Az `ENCODING` értéke UTF-8, ezért a magyar ékezetes karakterek is megfelelően kezelhetők. Az `indent-amount` azt adja meg, hogy a behúzás négy szóköz legyen.

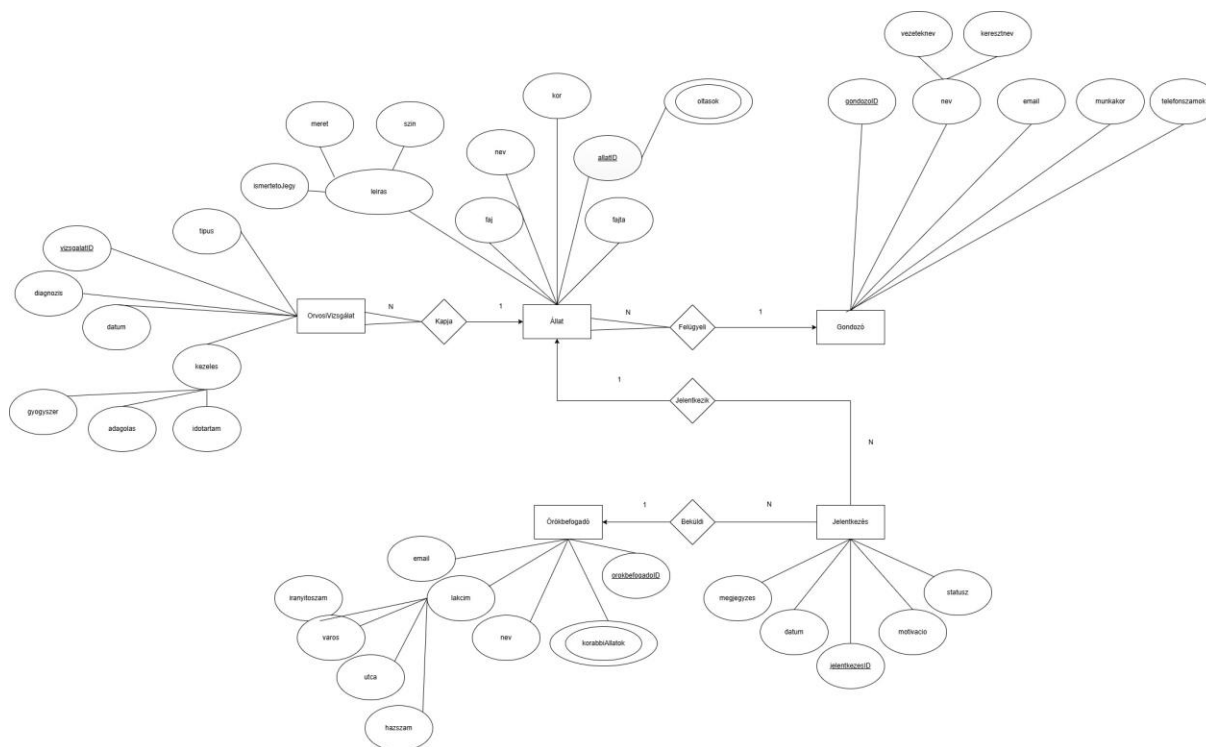
A mentés végül ezzel a résszel történik:

```
DOMSource source = new DOMSource(document);
StreamResult result = new StreamResult(outputFile);

transformer.transform(source, result);
```

A `DOMSource` a memóriában lévő DOM dokumentumot jelenti forrásként. A `StreamResult` a kimeneti fájlt jelöli ki. A `transform()` metódus elvégzi a tényleges mentést.

ER Modell dokumentáció



Az **ER modell** az állatmenhely nyilvántartó rendszer logikai adatbázis-tervét mutatja be **Peter Chen** jelölésrendszer alkalmazásával. A modell célja az állatmenhely működéséhez kapcsolódó

adatok strukturált tárolásának és kapcsolatainak szemléltetése. A rendszer fő feladata az állatok, gondozók, örökbefogadók, orvosi vizsgálatok és örökbefogadási jelentkezések adatainak kezelése.

A modell öt fő egyedet tartalmaz: **Állat**, **Gondozó**, **Örökbefogadó**, **OrvosiVizsgálat** és **Jelentkezés**. Az egyedek téglalappal kerültek jelölésre, az attribútumok ellipszissel, míg a kapcsolatok rombusz alakzattal jelennek meg. A kulcs attribútumok aláhúzással kerültek megkülönböztetésre.

Az **Állat** egyed tartalmazza az állatok alapadatait, például azonosítóját, nevét, fajtát, fajtáját és korát. Az állat egy összetett attribútummal is rendelkezik, amely a „leírás” nevet kapta. Ez további részekre bontható, például színre, méretre és ismertetőjegyre. Az Állat egyedhez többértékű attribútum is tartozik, amely az „oltások” mező. Ez azt szemlélteti, hogy egy állat több oltással is rendelkezhet.

A **Gondozó** egyed a menhely dolgozóinak adatait tárolja. Az attribútumok között szerepel a gondozó azonosítója, neve, email címe, munkaköre és telefonszámai. A név összetett attribútumként szerepel, amely vezetéknévre és keresztnévre bontható. A telefonszámok többértékű attribútumként jelennek meg, mivel egy gondozó több telefonszámmal is rendelkezhet.

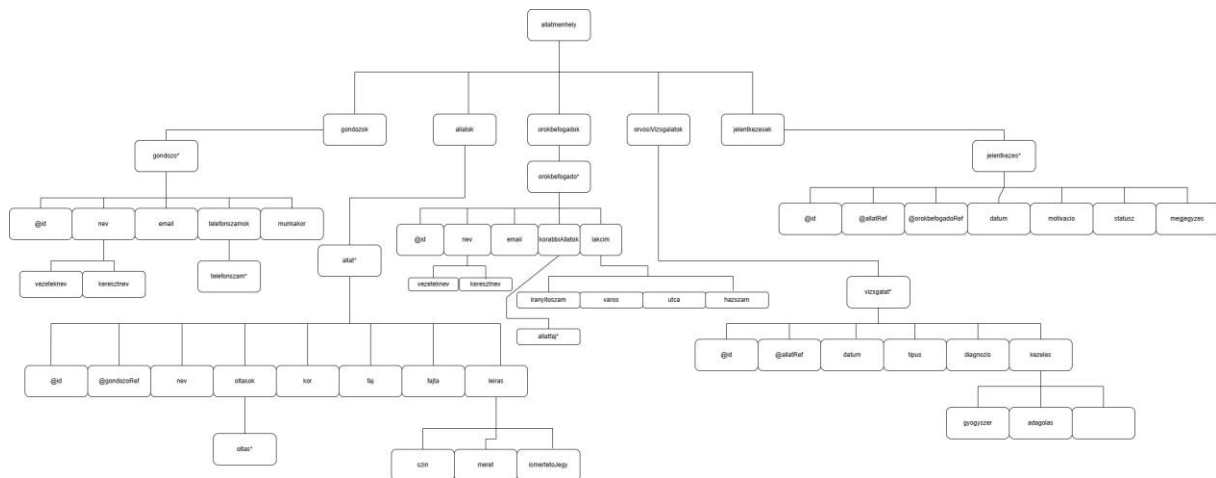
Az **Örökbefogadó** egyed az örökbefogadni kívánó személyek adatait tartalmazza. A modell tartalmazza az örökbefogadó azonosítóját, nevét, email címét, lakcímét és korábbi állatait. A lakcím összetett attribútum, amely irányítószámra, városra, utcára és házszámra bontható. A korábbi állatok attribútum többértékű, mivel egy személy korábban több állatot is tarthatott.

Az **OrvosiVizsgálat** egyed az állatokhoz kapcsolódó egészségügyi vizsgálatokat tárolja. Az attribútumok között szerepel a vizsgálat azonosítója, dátuma, típusa és diagnózisa. A „kezelés” attribútum összetett tulajdonságként jelenik meg, amely gyógyszerre, adagolásra és időtartamra bontható.

A **Jelentkezés** egyed az örökbefogadási folyamatot reprezentálja. Ez az egyed valójában egy kapcsolóegyed, amely az Állat és az Örökbefogadó közötti N:M kapcsolat felbontására szolgál. A kapcsolóegyed saját attribútumokkal rendelkezik, például jelentkezés azonosítóval, dátummal, motivációval, státusszal és megjegyzéssel. A modellben ez az elem teszi lehetővé az N:M kapcsolat attribútumainak kezelését.

A kapcsolatok között több 1:N kapcsolat is megtalálható. Egy Gondozó több Állatot felügyelhet, de egy állathoz egyszerre csak egy gondozó tartozik. Egy Állathoz több OrvosiVizsgálat kapcsolódhat, azonban egy vizsgálat kizárólag egy állathoz tartozik. Az Állat és az Örökbefogadó közötti kapcsolatot a Jelentkezés kapcsolóegyed valósítja meg.

XDM Modell dokumentáció



Az **XDM** modell az XML dokumentum hierarchikus szerkezetét mutatja be. A modell célja annak szemléltetése, hogy az XML fájl elemei milyen módon ágyazódnak egymásba, illetve hogyan épül fel a dokumentum fa struktúrája. Az XDM modell fontos szerepet játszik az XML alapú adatkezelésben, mivel segítségével jól áttekinthetővé válik a dokumentum logikai felépítése.

A dokumentum gyökéreleme az „**allatmenhely**” elem. Ez az elem foglalja össze a teljes rendszer összes adatát. A gyökérelem alatt öt fő adategység található: **gondozok**, **allatok**, **orokbefogadok**, **orvosiVizsgalatok** és **jelentkezesek**.

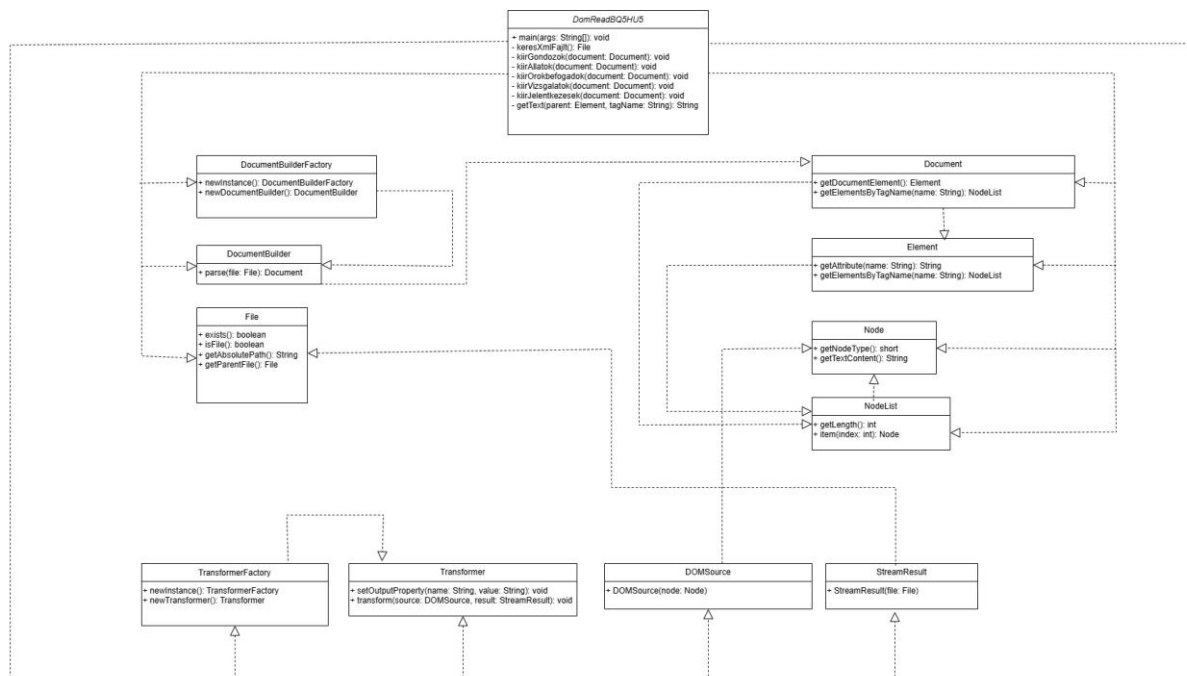
A **gondozok** elem alatt több gondozo elem szerepelhet, amit a „*” jelölés mutat. Egy gondozo elem tartalmazza az azonosítót, nevet, email címet, telefonszámokat és munkakört. A név összetett elemként szerepel, amely vezetéknévre és keresztnévre bontható. A telefonszámok elem alatt több telefonszam elem is megjelenhet, amely többértékű adatszerkezetet jelent.

Az **allatok** elem alatt több allat elem található. Egy allat elem rendelkezik azonosítóval és gondozó referencia attribútummal. Emellett tartalmazza az állat nevét, fajtát, fajtáját, korát, oltásait és leírását. A leiras elem összetett elemként működik, amely színre, méretre és ismertetőjegyre bontható. Az oltások elem alatt több oltas elem is szerepelhet.

Az **orokbefogadok** elem több orokbefogado elemet tartalmazhat. Az örökbefogadó elemekben szerepel az azonosító, név, email cím, korábbi állatok és lakcím. A lakcím összetett elemként jelenik meg, amely irányítószámra, városra, utcára és házszámra bontható. A **korábbiAllatok** elem alatt több allatfaj elem is található.

Az **orvosiVizsgalatok** rész több vizsgalat elemet tartalmazhat. Egy vizsgalat rendelkezik azonosítóval, állat referencia attribútummal, dátummal, típussal és diagnózissal. A kezeles elem összetett szerkezetű, amely gyógyszert, adagolást és időtartamot tartalmaz. A jelentkezesek elem alatt több jelentkezes elem jelenhet meg. Egy jelentkezés tartalmazza az azonosítót, az állat és az örökbefogadó referencia attribútumait, valamint a jelentkezés dátumát, motivációját, státuszát és megjegyzését. A „@” jelölés attribútumot, míg a „*” karakter ismétlődő elemet jelöl.

UML Diagram dokumentáció



Az **UML** osztálydiagram a Java alapú XML feldolgozó alkalmazás szerkezetét és a DOM API használatát mutatja be. A diagram célja annak szemléltetése, hogy a program milyen osztályokat használ az XML dokumentum beolvasására, feldolgozására és módosítására.

A rendszer központi eleme a **DomReadBQ5HU5** osztály, amely a teljes alkalmazás működését vezérli. Az osztály tartalmazza a **main** metódust, amely a program belépési pontja. A program indulásakor ez a metódus megkeresi az XML fájlt, majd létrehozza a DOM feldolgozáshoz szükséges objektumokat.

A **DomReadBQ5HU5** osztály több segédmetódust is tartalmaz. A **keresXmlFajlt** metódus az XML dokumentum megkereséséért felel. A **kiirGondozok**, **kiirAllatok**, **kiirOrokbefogadok**, **kiirVizsgalatok** és **kiirJelentkezesek** metódusok az XML dokumentum egyes részeit dolgozzák fel és jelenítik meg a konzolon. A **getText** segédmetódus egy adott XML elem szöveges tartalmának kiolvasását végzi.

A diagramon szerepelnek a DOM API legfontosabb osztályai is. **DocumentBuilderFactory** osztály felelős a **DocumentBuilder** objektum létrehozásáért. A **DocumentBuilder** végzi az XML fájl tényleges feldolgozását és Document objektummá alakítását.

A **Document** osztály az XML dokumentum teljes szerkezetét reprezentálja. Ennek segítségével érhető el a dokumentum gyökéreleme, valamint az egyes XML elemek listája. Az **Element** osztály az XML elemek kezelésére szolgál, például attribútumok és gyermekelemek elérésére.

A **Node** és **NodeList** osztályok a DOM fa szerkezetének kezelését biztosítják. A Node egy XML csomópontot reprezentál, míg a NodeList több csomópont gyűjteményét tárolja.

A **TransformerFactory** és **Transformer** osztályok a dokumentum módosítás utáni visszairásért felelősek. A Transformer segítségével a program képes az XML dokumentumot megfelelő formázással visszamenteni.

A formázást szolgálja a **setOutputProperty** metódus, amely a behúzás mértékét és a dokumentum olvashatóságát állítja be. A **DOMSource** és **StreamResult** osztályok a dokumentum mentésében vesznek részt. A **DOMSource** a DOM struktúrát reprezentálja, míg a **StreamResult** a fájlba történő kimenetet kezeli. Az UML diagram dependency kapcsolatokat használ, amelyeket szaggatott vonal és üres nyílhegy jelöl. Ezek a kapcsolatok azt mutatják, hogy az egyes osztályok milyen más osztályokat használnak működésük során. A diagram jól szemlélteti a Java DOM API és a saját alkalmazás kapcsolatát, valamint a program objektumorientált felépítését.

XML Dokumentáció

Az XML dokumentum az állatmenhely nyilvántartó rendszer adatainak tárolására szolgál. Az XML célja, hogy strukturált, hierarchikus formában tárolja a rendszerhez kapcsolódó adatokat, amelyeket a Java alapú DOM feldolgozó program képes beolvasni és feldolgozni.

A dokumentum gyökéreleme az **allatmenhely**, amely a teljes rendszer összes adatát összefogja. A gyökérelem alatt öt fő adategység található: **gondozok**, **allatok**, **orokbefogadok**, **orvosiVizsgalatok** és **jelentkezesek**. A **gondozok** elem a menhely dolgozóinak adatait tartalmazza. Minden **gondozo** elem rendelkezik egy egyedi id attribútummal. A gondozó adatai között szerepel a név, email cím, munkakör és telefonszámok listája. A név összetett szerkezetű, mivel külön tárolásra kerül a vezetéknev és a keresztnév.

Az **allatok** rész az állatok adatait tartalmazza. Az egyes **allat** elemek rendelkeznek saját azonosítóval és egy **gondozoRef** attribútummal, amely a gondozó azonosítójára hivatkozik. Az állat adatain belül szerepel a név, faj, fajta, kor, oltások és a leírás. A leírás további részekre bontható, például színre, méretre és ismertetőjegyre. Az **orokbefogadok** rész az örökbefogadni kívánó személyek adatait tárolja. Az örökbefogadó rendelkezik saját azonosítóval, névvel, email címmel, lakcímmel és korábbi állatok listájával. A lakcím összetett szerkezetű elem, amely irányítószámot, várost, utcát és házszámot tartalmaz.

Az **orvosiVizsgalatok** elem az állatok egészségügyi vizsgálatait tárolja. A **vizsgalat** elem tartalmazza az azonosítót, az állat referencia attribútumát, a vizsgálat dátumát, típusát, diagnózisát és a kezelési adatokat. A kezelés további részekre bontható, például gyógyszerre, adagolásra és időtartamra. A **jelentkezesek** rész az örökbefogadási folyamat jelentkezéseit tartalmazza. A **jelentkezes** elem tárolja az állat és az örökbefogadó referencia attribútumait, valamint a jelentkezés dátumát, motivációját, státuszát és megjegyzését.

Az XML dokumentum egyik legfontosabb előnye a jól strukturált hierarchikus felépítés, amely lehetővé teszi az adatok könnyű feldolgozását DOM API segítségével. A dokumentum támogatja az összetett elemeket, az attribútumokat és az ismétlődő elemek kezelését is.

A Java program a DOM parser segítségével dolgozza fel az XML dokumentumot. A program képes az XML elemek és attribútumok beolvasására, módosítására, valamint a dokumentum formázott visszamentésére.

XSD Dokumentáció

Az XSD séma az XML dokumentum szerkezetének és adattípusainak ellenőrzésére szolgál. Az XSD célja annak biztosítása, hogy az XML dokumentum megfeleljen az előre meghatározott szabályoknak és struktúrának. Az XSD dokumentum meghatározza az XML fájlban szereplő elemeket, attribútumokat, azok sorrendjét, adattípusait és előfordulási számát. A séma segítségével ellenőrizhető, hogy az XML dokumentum szerkezetileg helyes-e.

A séma gyökéreleme az **xs:schema**, amely az XML Schema szabvány névterét használja. A fő elem az **allatmenhely**, amely komplex típusként került definiálásra. Ez a komplex típus tartalmazza a rendszer összes fő elemét. Az XSD külön kezeli az egyszerű és összetett típusokat. Az összetett típusok lehetővé teszik gyermekelemek és attribútumok definiálását. Ilyen például a **gondozo**, **allat**, **orokbefogado**, **vizsgalat** és **jelentkezes** elem. Az attribútumok az **xs:attribute** elem segítségével kerülnek definiálásra. Az azonosítók általában **xs:string** típust használnak, míg egyes numerikus mezők **xs:int** típusként kerülnek meghatározásra.

A séma használ **minOccurs** és **maxOccurs** attribútumokat is. Ezek szabályozzák, hogy egy elem hányszor fordulhat elő az XML dokumentumban. A **maxOccurs="unbounded"** lehetővé teszi több elem ismétlődését, például több állat vagy több jelentkezés tárolását. Az XSD séma különösen fontos az XML validáció során. A Java program képes az XML dokumentumot az XSD alapján ellenőrizni, így kiszűrhetők a hibás vagy hiányos adatok. Ez biztosítja az adatok konzisztenciáját és megbízhatóságát. A séma támogatja az összetett struktúrákat is. Például a név vagy a lakcím további részekre bontható. Ez lehetővé teszi az adatok pontosabb és strukturáltabb tárolását. Az XSD dokumentum fontos szerepet játszik az XML alapú rendszerekben, mivel szabványos módon definiálja az adatszerkezetet és biztosítja a dokumentumok helyességét.